A Comprehensive Dossier on Stochastic Reliability Modeling and Decision-Making Control for NVIDIA H100 GPU Clusters

The rapid evolution of large language model (LLM) architectures toward trillion-parameter scales has transformed the underlying computational requirements into a critical reliability engineering challenge. Modern AI supercomputers, comprised of tens of thousands of NVIDIA H100 and A100 GPUs, operate at a level of complexity where hardware failures are no longer occasional exceptions but are instead an inherent property of the system's operational state.[1] Managing these environments requires a transition from reactive maintenance toward proactive, stochastic control frameworks. This dossier provides an exhaustive collection of empirical data, modeling frameworks, and algorithmic strategies to support the implementation of probabilistic learning models, including Discrete-Time Markov Chains (DTMC), Continuous-Time Markov Chains (CTMC), Hidden Markov Models (HMM), and Reinforcement Learning (RL) agents for cluster management.[5]

Empirical Reliability Facts and Hardware Dynamics

The reliability of high-density GPU clusters is governed by the intersection of thermal stress, power delivery constraints, and high-frequency communication patterns unique to deep learning workloads.[9] Traditional high-performance computing metrics often fail to capture the nuances of LLM training, where gang-scheduled jobs render the entire system vulnerable to the failure of a single component.[4]

Large-Scale Failure Observations

Analysis of production-scale environments, specifically the 54-day training period of the Llama 3 405B model involving 16,384 NVIDIA H100 GPUs, revealed a total of 466 job interruptions.[1] Out of these, 419 were unexpected failures, resulting in a mean time to failure (MTTF) of approximately three hours.[1] This high frequency is attributed to the fact that over half of the failures are directly related to the GPUs or their onboard HBM3 memory subsystems.[1]

The scaling laws of reliability indicate that as cluster size increases, the risk of failure grows proportionally, creating a significant impact on "goodput"—the ratio of useful compute time to total wall-clock time.[10]

Cluster Scale	Mean Time To	Failure Rate per	Reliability

(GPUs)	Failure (MTTF)	Node-Day	Benchmark Context
1,024	7.9 Hours to 3.66 Days	~2.5 - 7.76	Baseline for modern H100 training [10]
16,384	1.8 Hours to 3.0 Hours	~7.76 - 13.3	Representative of foundation model pre-training [1]
131,072	14 Minutes	~17.5 (Peak)	Projected limit for next-generation clusters [10]

The economic cost of these failures is substantial. For a 1,024-GPU cluster, improving the total average model FLOPs utilization (MFU) from typical levels of 42% to an optimized 51% can save nearly a week of training time on a month-long job, which equates to over $12,600 in saved compute costs based on standard GPU-hour pricing.[14]

Failure Mode Taxonomy and Severity

A hierarchical classification system for GPU failures is essential for defining state transitions in stochastic models.[9] Research identifying patterns across 1,250 GPUs over 18 months highlights that thermal events and power delivery issues dominate the failure landscape, driven by the sustained 700W power draw of the H100 architecture.[9]

Failure Category	Distribution (%)	Mechanism of Failure	Symptoms for Observation
Thermal Events	31%	Degradation of thermal interface materials; inlet air > 24°C [9]	Thermal throttling; frequency oscillation [9]
Power Delivery	22%	Voltage regulation instability during memory write spikes [9]	Transient load failures; power network degradation [9]
Memory Subsystem	18%	HBM3/HBM2e	ECC page

		sensitivity; cumulative single-bit errors [9]	retirement; row remapping; DBE events [15]
Silicon/Manufacturing	13%	Electromigration; bonding failures; early operational life defects [9]	Instant hardware failure; "falling off bus" [9]
Firmware/Software	10%	Driver bugs; resource leaks; timing-sensitive software traps [9]	Kernel panics; XID codes 38, 43 [15]
Interconnect/Link	6%	NVLink or InfiniBand cable/switch material failures [17]	Flapping links; NCCL timeouts [17]

Candidate State Spaces for Stochastic Modeling

The efficacy of a reliability model is contingent upon its ability to represent the physical condition of the hardware through a discrete or continuous state space.[5]

Operational Health States (DTMC/CTMC/MDP)

For models focused on decision-making, such as Markov Decision Processes (MDP) or Reinforcement Learning (RL), the state space must capture the functional capacity of the GPU.[5]

1. **State S_H (Healthy):** The GPU is operating within nominal specifications. Telemetry shows standard power draw, stable clock speeds, and zero ECC errors.[18]
2. **State S_D (Degraded):** The system exhibits non-critical symptoms such as minor thermal throttling or single-bit ECC errors (SBE). Performance is slightly reduced, but the training job remains active.[9]
3. **State S_M (Maintenance-Required):** The device has accumulated enough soft errors to exceed safety thresholds (e.g., high row-remapping rate). This state precedes failure and is often referred to as the "lemon node" phase.[16]
4. **State S_F (Failed-Recoverable):** A critical error has occurred, such as a double-bit ECC error (DBE) or an XID 48/79 event. The system requires a software-level intervention, such as a driver reset or node reboot.[15]
5. **State S_R (Failed-Permanent):** Hardware damage has occurred to the silicon, HBM memory, or interconnects. This state requires physical part replacement and node

replacement in the scheduler.[10]

Hidden and Latent States (HMM/POMDP)

In many cases, the true health of a GPU is a latent variable that must be inferred from noisy observables.[7]

- **Belief State $b(s)$:** In a Partially Observable Markov Decision Process (POMDP), the agent maintains a probability distribution over states based on historical telemetry and current observations.[7]
- **Latent Anomaly States:** These include "Silent Data Corruption" (SDC) risks and "Hidden Performance Degradation" where no explicit error code is thrown, but numerical stability is compromised.[2]
- **Positional Vulnerability:** States can be augmented with environmental metadata, such as the rack position (upper third vs. lower third) to adjust transition probabilities based on known higher failure rates in high-heat zones.[9]

Probabilistic Modeling Assumptions

Building a robust mathematical framework for GPU cluster reliability requires specific assumptions regarding the distribution of failures and the nature of transitions.[18]

The Markov Property and Stationary Distributions

The foundational assumption for DTMC and CTMC models is the Markov property, asserting that the probability of moving to a future state depends only on the current state and not the history of how the system arrived there.[23] While memorylessness is mathematically convenient, empirical data on GPU aging and electromigration suggest that transition rates may change over time, necessitating the use of non-stationary models or segmented Markov chains.[18]

For a system with a stationary distribution π, the long-term behavior of the cluster satisfies:

$$\pi = \pi P$$

where P is the transition matrix.[24] This allows researchers to project the steady-state availability of a cluster given a specific maintenance cadence.

Failure Distributions: Weibull vs. Exponential

Standard reliability modeling often defaults to an exponential distribution, which assumes a constant failure rate.[18] However, data from high-performance GPU clusters show that a Weibull distribution with a decreasing hazard rate ($k < 1$) often provides a better fit, particularly for

capturing "infant mortality" and burn-in periods in large-scale deployments.[25]

The transition rate $\lambda(t)$ in a Weibull-based reliability model is:

$$\lambda(t) = \frac{k}{\lambda}\left(\frac{t}{\lambda}\right)^{k-1}$$

This assumption is critical for HMM-based failure prediction, as it informs the likelihood that a "Degraded" state will transition to a "Failed" state based on the age or cumulative utilization of the component.[25]

Uncertainty in Transition Rates

Given the limited availability of public failure datasets for H100 systems, transition rates between "Normal," "Defective," and "Failed" states are often modeled with uncertainty.[18] A multi-phase Markov model assuming that transition rates follow a Beta distribution—defined by known upper and lower bounds from similar historical facilities—provides a more realistic estimation of system unavailability.[18]

Data Sources and Observables

The implementation of any probabilistic learning model for GPU clusters relies on a high-fidelity telemetry pipeline.[31] Modern systems generate 50x more telemetry data than traditional CPU infrastructure, requiring specialized ingestion and analysis.[32]

NVIDIA Data Center GPU Manager (DCGM)

DCGM serves as the primary source of ground-truth telemetry. It provides over 100 metrics that are essential for HMM emission probabilities and MDP state definitions.[31]

Metric Group	Observable Fields	Diagnostic Value for Modeling
Utilization	SM_ACTIVE, TENSOR_ACTIVE, MEM_COPY_UTIL	Defines load-dependent transition probabilities [31]
Memory Health	ECC_SBE_VOLATILE, ECC_DBE_VOLATILE, RETIRED_PAGES	Primary indicators for degradation and uncorrectable memory failure [10]

Power/Thermal	GPU_TEMP, POWER_USAGE, CLOCKS_THROTTLE_REASONS	Predictors for thermal-induced state transitions and power spikes [9]
Communications	NVLINK_RX_BYTES, NVLINK_TX_BYTES, IB_LINK_STATUS	Used to detect fabric-level failures and "straggler" nodes [1]
Exceptions	XID_ERRORS, SXid_ERRORS	Discrete error codes representing specific failure events [10]

System-Level Logs and Event Streams

While DCGM provides hardware-level data, system-level logs provide the context for recovery and repair actions.[16]

- **XID Error Catalog:** XID messages are reports from the NVIDIA driver printed to the kernel log. They indicate whether an error is software-related (e.g., XID 13/31), hardware-related (e.g., XID 79 "fallen off bus"), or bus-related (e.g., XID 32 DMA controller error).[15]
- **NCCL Flight Recorder:** This tool records collective communication metadata into a circular buffer, allowing for rapid diagnosis of hangups during large-scale training jobs.[1]
- **Job Scheduler Metadata:** Slurm or Kubernetes logs provide the "Goodput" and "Total Idle Time" metrics, which serve as the reward signals for RL-based scheduling agents.[10]

Algorithms Relevance and Stochastic Decision Making

The selection of algorithms for GPU cluster control is driven by the need to balance performance maximization against the risk of catastrophic failure.[6]

Markov Decision Processes and Resource Scheduling

The GPU resource scheduling problem is frequently modeled as an MDP where the action space involves regional allocation decisions or node-level task assignments.[6]

The standard reward function R_t in these environments integrates throughput, cost, and stability:

$$R_t = -(w_1 \cdot \text{ResponseTime} + w_2 \cdot \text{LoadVariance} + w_3 \cdot \text{OperationalCost})$$

Optimal transport (OT) theory can be combined with MDPs to coordinate inter-region task

distribution, significantly reducing latency and switching costs compared to myopic single-timeslot strategies.[6]

Reinforcement Learning (RL) for Proactive Control

RL agents, particularly those using Proximal Policy Optimization (PPO), are deployed to learn multi-time-slot allocation strategies.[6]

- **TORTA Architecture:** The Temporal Optimal Resource scheduling via Two-layer Architecture (TORTA) integrates spatial and temporal dimensions.[6] The macro-level scheduler handles task distribution across regions, while the micro-level allocator refines task-to-server assignments within each region.[6]
- **Smoothness Rewards:** To avoid the high overhead of frequent server state transitions, RL policies often include a smoothness reward $r_{smooth,t} = -|

|A_{RL,t} - A_{RL,t-1}||_F^2$, which penalizes oscillations in resource allocation.[40]

- **Condition-Based Maintenance:** RL can be used to optimize maintenance schedules by balancing the "risk of failure" against the "reward of extended operating hours," often achieved through Q-learning or SARSA in discrete state spaces.[5]

Hidden Markov Models for Predictive Maintenance

HMMs are particularly relevant for identifying the transition from a "Healthy" state to a "Degraded" state that is not yet visible to the scheduler.[7]

- **Failure Time Prediction:** Forward probability calculations in an HMM allow for the prediction of failure time distributions from historical and real-time DCGM data.[7]
- **Baum-Welch and Forward Algorithms:** These are used to iteratively refine the transition (A) and emission (B) matrices as more telemetry data is ingested, improving predictive accuracy over time.[20]
- **Anomaly Detection:** HMMs can provide a "data-driven early-warning signal" up to 5 minutes before failure by detecting local temporal patterns such as latency spikes or bursts of soft errors.[43]

Implementation-Ready Parameters and Formulas

For practitioners building these models, the following parameters from industry-leading studies on 16k+ GPU clusters provide a baseline for initialization and validation.[4]

Reliability Projections and Scale Formulas

As clusters grow, the probability of job completion without failure can be estimated using the Mean Time to Failure of the job ($MTTF_{job}$).[4]

The expected Effective Training Time Ratio (ETTR) is modeled as:

$$E \approx 1 - N_{nodes} \cdot r_f \left(u_0 + \frac{\Delta t_{cp}}{2} \right)$$

where:

- N_{nodes} is the number of nodes in the job.[4]
- r_f is the failure rate per node-day.[4]
- u_0 is the initialization and recovery time (e.g., loading checkpoints).[4]
- Δt_{cp} is the time between checkpoints.[4]

Parameter	Recommended Initial Value	Origin/Source
Checkpoint Penalty (w_{cp})	17s (Async) to 129s (Sync)	CoreWeave H100 benchmarks [12]
Recovery Time Loss	Dozens of minutes to hours	Depending on automation level [10]
Failure Prediction Accuracy	81% (Precision)	H100/A100 failure model [9]
Predictive Lead Time	4 - 24 Hours	Proactive monitoring window [9]
Power-Capping Impact	1-2% throughput fluctuation	Due to DVFS and temperature [1]

Reward Function Weightings for TORTA (RL/MDP)

In the TORTA framework for resource scheduling, the reward weights are tuned to balance latency and cost.[6]

- **Latency Weight (w_1):** Prioritizes minimizing the network delay between tasks and the data source.[6]
- **Bandwidth Weight (w_2):** Penalizes tasks that require excessive inter-region data movement.[8]

- **Operational Cost Weight (w_3):** Captures energy pricing and power-capping costs.[8]

The load balance coefficient (LB) is typically targeted at 0.74 - 0.76 for optimal efficiency across geographical regions.[6]

Operational Observations and Strategic Recovery

A comprehensive reliability dossier must account for the specific mechanisms used to recover from failures once they are detected.[4]

Checkpointing Dynamics

Checkpointing is the primary fault-tolerance mechanism for LLM training. Synchronous memory writes across multiple GPUs during checkpointing create a distinct "power signature" that can trigger regulator failures.[9] Implementing asynchronous checkpointing using tools like Tensorizer can reduce the save time from 129 seconds to 17 seconds, maintaining 99%+ compute utilization despite the overhead.[14]

Effective Training Time Ratio (ETTR) and Goodput

The business impact of reliability is measured through Goodput, defined as:

$$Goodput = \frac{\text{Useful Compute Time}}{\text{Useful Compute Time} + \text{Idle Compute Time}}$$

The idle compute time consists of checkpointing time, lost training time from the last checkpoint, and recovery time (node replacement and job restart).[10] Production-grade H100 infrastructures aim for an ETTR of 97.5% at the 1,024-GPU scale, though typical reports for foundation models often cite figures as low as 90% or 35–45% MFU.[10]

The "Straggler" and "Lemon" Node Identification

A critical task for HMMs and anomaly detection is identifying "lemon nodes"—GPUs that pass basic health checks but exhibit persistent performance degradation or flapping behavior.[17]

- **Flapping Links:** InfiniBand links that transition rapidly between up and down states can disrupt the entire communication fabric, necessitating adaptive routing or the isolation of the offending node.[17]
- **GSP Timeouts:** Code regressions in GPU drivers (GSP timeouts) have historically caused spikes in failure rates, appearing in telemetry as non-deterministic hangups that require driver patches to resolve.[17]

Implementation Strategy for Probabilistic Learning

Models

The integration of the data and models provided in this dossier should follow a structured progression to ensure cluster stability.

1. **Phase 1: Telemetry Baseline (Observation).** Implement DCGM Exporter and Prometheus scraping to build a historical database of XID codes and SM utilization.[31]
2. **Phase 2: State Estimation (HMM).** Define the hidden states ($Healthy$, $Degraded$, $Failed$) and use the forward algorithm to estimate the probability of failure based on current telemetry streams.[7]
3. **Phase 3: Reward Formulation (MDP).** Establish the weights (w_1, w_2, w_3) for the job scheduler, incorporating the cost of downtime and the benefits of checkpointing.[6]
4. **Phase 4: Policy Optimization (RL).** Train PPO-based agents in a simulated environment using the Weibull failure distributions and projected MTTF parameters before deploying to the production cluster.[6]

By synthesizing hardware-level failure modes with high-level probabilistic modeling, researchers and infrastructure engineers can transform the "inevitability of failure" into a manageable, stochastic control problem, ensuring the efficient training of next-generation AI models.[2]

Works cited

1. Meta reports that half of the failures during Llama 3 traini... - moomoo Community, accessed April 2, 2026, https://www.moomoo.com/community/feed/meta-reports-that-half-of-the-failures-during-llama-3-112873025896854
2. Faulty Nvidia H100 GPUs and HBM3 memory caused half of failures during LLama 3 training - Tom's Hardware, accessed April 2, 2026, https://www.tomshardware.com/tech-industry/artificial-intelligence/faulty-nvidia-h100-gpus-and-hbm3-memory-caused-half-of-the-failures-during-llama-3-training-one-failure-every-three-hours-for-metas-16384-gpu-training-cluster
3. MoFa: A Unified Performance Modeling Framework for LLM Pretraining - arXiv, accessed April 2, 2026, https://arxiv.org/html/2511.09837v2
4. Revisiting Reliability in Large-Scale Machine Learning Research Clusters - arXiv, accessed April 2, 2026, https://arxiv.org/pdf/2410.21680
5. Markov Decision Process (MDP) in Reinforcement Learning - GeeksforGeeks, accessed April 2, 2026, https://www.geeksforgeeks.org/machine-learning/what-is-markov-decision-process-mdp-and-its-relevance-to-reinforcement-learning/
6. Temporal-Aware GPU Resource Allocation for Distributed LLM Inference via Reinforcement Learning - arXiv, accessed April 2, 2026,

https://arxiv.org/html/2507.10259v2
7. Hidden Markov Models and Their Application for Predicting Failure Events - PMC, accessed April 2, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC7304047/
8. Cross-Timeslot Optimization for Distributed GPU Inference Using Reinforcement Learning, accessed April 2, 2026, https://ics.uci.edu/~habibif/cross-timeslot.pdf
9. Sarcouncil Journal of Engineering and Computer Sciences GPU Reliability in AI Clusters: A Study of Failure Modes and Effects, accessed April 2, 2026, https://sarcouncil.com/download-article/SJECS-97-2025-298-306.pdf
10. Fault-tolerant training: How we build reliable clusters for distributed AI workloads - Nebius, accessed April 2, 2026, https://nebius.com/blog/posts/how-we-build-reliable-clusters
11. Llama 3.1 training fails once every three hours on average. The H1 million-GPU cluster is very fragile, and temperature fluctuations can affect throughput. - EEWorld, accessed April 2, 2026, https://en.eeworld.com.cn/mp/QbitAI/a382499.jspx
12. CoreWeave Training Benchmarks Whitepaper August 2025, accessed April 2, 2026, https://cdn.prod.website-files.com/62bc66d283fd9c34ffec780a/689fa33a0e99f19c11c8ecbe_CoreWeave%20Training%20Benchmarks%20Whitepaper%20August%202025.pdf
13. Lazarus: Resilient and Elastic Training of Mixture-of-Experts Models - arXiv, accessed April 2, 2026, https://arxiv.org/html/2407.04656v2
14. NVIDIA H100 Benchmarks for Large-Scale Training | CoreWeave, accessed April 2, 2026, https://www.coreweave.com/blog/nvidia-h100-gpu-benchmark-results-what-we-learned-from-large-scale-gpu-testing
15. How to handle common Xid events--Machine Learning Platform-Byteplus, accessed April 2, 2026, https://docs.byteplus.com/en/docs/mlp/How_to_handle_common_Xid_events
16. GPU Health | Modal Docs, accessed April 2, 2026, https://modal.com/docs/guide/gpu-health
17. Revisiting Reliability in Large-Scale Machine Learning Research Clusters - arXiv, accessed April 2, 2026, https://arxiv.org/html/2410.21680v1
18. Performance Analysis for a Degrading System with Markov Model - ResearchGate, accessed April 2, 2026, https://www.researchgate.net/publication/363336255_Performance_analysis_for_a_degrading_system_with_Markov_model
19. Fire-Flyer AI-HPC: A Cost-Effective Software-Hardware Co-Design for Deep Learning - arXiv, accessed April 2, 2026, https://arxiv.org/html/2408.14158v1
20. A Dynamic Hidden Markov Model with Real-Time Updates for Multi-Risk Meteorological Forecasting in Offshore Wind Power - MDPI, accessed April 2, 2026, https://www.mdpi.com/2071-1050/17/8/3606
21. Network motif detection using hidden markov models - PMC, accessed April 2, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC12647810/
22. Understanding Data Movement and Processing Platform at Netflix -

MarkTechPost, accessed April 2, 2026, https://www.marktechpost.com/2022/09/02/understanding-data-movement-and-processing-platform-at-netflix/
23. Markov Chain - Lark, accessed April 2, 2026, https://www.larksuite.com/en_us/topics/ai-glossary/markov-chain
24. Pre-trained Large Language Models Learn to Predict Hidden Markov Models In-context - OpenReview, accessed April 2, 2026, https://openreview.net/pdf/d9c6c3a5ef2b457a99d3851f69c626f9896663ed.pdf
25. Understanding GPU errors on large-scale HPC systems and the implications for system design and operation | Request PDF - ResearchGate, accessed April 2, 2026, https://www.researchgate.net/publication/282377434_Understanding_GPU_errors_on_large-scale_HPC_systems_and_the_implications_for_system_design_and_operation
26. Efficient Probabilistic Model Checking on General Purpose Graphics Processors - Spin, accessed April 2, 2026, https://spinroot.com/spin/symposia/ws09/Bosnacki.pdf
27. SuperBench: Improving Cloud AI Infrastructure Reliability with Proactive Validation, accessed April 2, 2026, https://arxiv.org/html/2402.06194v2
28. SuperBench: Improving Cloud AI Infrastructure Reliability with Proactive Validation - USENIX, accessed April 2, 2026, https://www.usenix.org/system/files/atc24-xiong.pdf
29. A large-scale study of soft-errors on GPUs in the field - ResearchGate, accessed April 2, 2026, https://www.researchgate.net/publication/299641571_A_large-scale_study_of_soft-errors_on_GPUs_in_the_field
30. here - The Education University of Hong Kong, accessed April 2, 2026, https://www.eduhk.hk/eac-isba2024/BOA_0622.pdf
31. NVIDIA DCGM, accessed April 2, 2026, https://developer.nvidia.com/dcgm
32. GPU Cluster Monitoring: Real-Time Performance Analytics and Predictive Maintenance, accessed April 2, 2026, https://introl.com/blog/gpu-cluster-monitoring-real-time-analytics-predictive-maintenance
33. Collect and view DCGM metrics | Google Kubernetes Engine (GKE), accessed April 2, 2026, https://docs.cloud.google.com/kubernetes-engine/docs/how-to/dcgm-metrics
34. NVIDIA DCGM Monitoring: Setup, Metrics & Alerts | MetricFire, accessed April 2, 2026, https://www.metricfire.com/blog/why-gpu-monitoring-matters-tracking-utilization-power-and-errors-with-dcgm/
35. DCGM Release Notes - NVIDIA Documentation, accessed April 2, 2026, https://docs.nvidia.com/datacenter/dcgm/latest/release-notes/changelog.html
36. XID Errors - NVIDIA Documentation, accessed April 2, 2026, https://docs.nvidia.com/deploy/pdf/XID_Errors.pdf
37. Working with Xid Errors - NVIDIA Documentation, accessed April 2, 2026,

https://docs.nvidia.com/deploy/xid-errors/working-with-xid-errors.html
38. Algorithmic Techniques for GPU Scheduling: A Comprehensive Survey - Preprints.org, accessed April 2, 2026, https://www.preprints.org/manuscript/202505.0152/download/final_file
39. Deep Reinforcement Learning for Job Scheduling and Resource Management in Cloud Computing: An Algorithm-Level Review - arXiv, accessed April 2, 2026, https://arxiv.org/html/2501.01007v1
40. Temporal-Aware GPU Resource Allocation for Distributed LLM Inference via Reinforcement Learning - arXiv, accessed April 2, 2026, https://arxiv.org/pdf/2507.10259
41. Condition-Based Maintenance scheduling of an aircraft fleet under partial observability - AGIFORS, accessed April 2, 2026, https://www.agifors.org/resources/Documents/Anna%20Valicek%20Papers/2024/04-Iordanis%20Tseremoglou%20-%20Paper%20-%20Condition-Based%20Maintenance%20scheduling%20of%20an%20aircraft%20fleet.pdf
42. Joint Optimization of Time-Aware Condition-Based Maintenance and Repair Resource Management for Gantry Crane Clusters Based on I - IEEE Xplore, accessed April 2, 2026, https://ieeexplore.ieee.org/iel8/6287639/10380310/10788675.pdf
43. Apostolos Kokolis's research works | Meta and other places - ResearchGate, accessed April 2, 2026, https://www.researchgate.net/scientific-contributions/Apostolos-Kokolis-2155332310
44. Predicting GPU Failures With High Precision Under Deep Learning Workloads - Cheng Tan, accessed April 2, 2026, https://naizhengtan.github.io/doc/papers/liu23predicting.pdf
45. Nvidia DCGM Exporter - Datadog Docs, accessed April 2, 2026, https://docs.datadoghq.com/integrations/dcgm/